Niching Subset Simulation: an Implementation Guide and User Manual

Companion notes to the single-file implementation

15 October 2026

Abstract

These notes document a single Python program that estimates small failure probabilities $p_F = P\big(g(\mathbf{x}) \geq Y_F\big)$, $\mathbf{x} \sim N(\mathbf{0}, \mathbf{I}_d)$, with two methods side by side: a vectorised implementation of subset simulation (SuS) in the sense of Au and Beck [1], and Kinnear's niching subset simulation (NSuS) [2], copied verbatim from the authors' companion repository. We describe every component of the code, state the mathematics each component implements, explain every figure the program produces, and show how to change the performance function and the algorithmic parameters for experiments. Numerical results are reported for the ring problem $g(\mathbf{x}) = x_1^2 + x_2^2$ and for the piecewise linear example of [2].

Contents

1 Introduction **3**
1.1 Layout of the program . . . 3

2 Problem set-up **3**
2.1 Parameters . . . 3
2.2 The performance function . . . 4

3 Subset simulation **4**
3.1 Principle . . . 4
3.2 Algorithm . . . 4
3.3 Estimate and cost . . . 5

4 The building blocks of NSuS **5**
4.1 Cached performance function (`performance_function.py`) . . . 5
4.2 Samples and indicators (`level_create.py`, `indicator.py`) . . . 5
4.3 Markov chain (`markov_chain.py`) . . . 5
4.4 Buds (`bud_create.py`) . . . 6
4.5 Levels (`level.py`) . . . 6
4.6 Allocation, choice and stopping . . . 6

5 The main loop **6**

6 The niching step **7**

7 Estimation **8**
7.1 The estimator . . . 8
7.2 Coefficient of variation . . . 9
7.3 The two assembled algorithms . . . 9

8 Figures **9**

9 The run block and its output **12**

10 Changing the problem and the parameters **12**
10.1 Parameters 12
10.2 Performance functions 12
10.3 Figures for a new problem 13

11 Numerical experiments **14**
11.1 Ring, $Y_F = 20$, $p_F = 4.54 \times 10^{-5}$ 14
11.2 Piecewise linear, $Y_F = 0$, $p_F = 3.18 \times 10^{-5}$ 14

12 Troubleshooting **15**

1 Introduction

Let $\mathbf{x} = (x_1, \ldots, x_d)$ have independent standard normal components, let $g : \mathbb{R}^d \to \mathbb{R}$ be a *performance function* and Y_F a *critical threshold*. The *failure domain* and *failure probability* are

$$F = \{\mathbf{x} \in \mathbb{R}^d : g(\mathbf{x}) \geq Y_F\}, \qquad p_F = P(\mathbf{x} \in F) = \int_{\mathbb{R}^d} \mathbb{1}_F(\mathbf{x})\, \varphi_d(\mathbf{x})\, d\mathbf{x}, \tag{1}$$

where φ_d is the d-dimensional standard normal density. When p_F is of order 10^{-4} or smaller, crude Monte Carlo needs roughly $10/p_F$ evaluations of g to observe ten failures. Subset simulation reduces this cost by writing F as the last of a nested sequence of more probable events. Its weakness is that the sequence follows a *single* path; if F consists of several separate regions, regions that look unpromising early may be lost. Niching subset simulation grows a *tree* of levels instead, and follows each region (*niche*) on its own branch.

1.1 Layout of the program

The program is one Python file. Table 1 lists its parts in file order and their origin.

Part	Content	Origin
Imports	numerical, graph, machine-learning and plotting libraries	both
Parameters	d, Y_F, n, p, `SEED`, `SAVE_PLOT`, `GRAPH_SIZE`, `MAX_BRANCHES`, `SCALE`	user
`performance_function`	g; currently $g(\mathbf{x}) = x_1^2 + x_2^2$	user
`subset_simulation`	subset simulation, Section 3	user
`verbose` … `HillValleyGraphNSuS`	the NSuS package, Sections 4–7	Kinnear, verbatim
`plot_levels_2d`	Figure 2	user
`plot_nsus_levels_2d`, `plot_branch_tree`	Figures 3, 4	added
`contour_plot`, `partitioner_figures`	Figures 5–6	Kinnear's figure scripts
run block	runs both methods, prints, plots	user, plus Kinnear's example

Table 1: The parts of the program.

2 Problem set-up

2.1 Parameters

```
d = 2              # dimension of the input space
YF = 20.0          # critical threshold (failure <=> g(x) > YF)
n = 3000           # number of samples per level
p = 0.1            # level probability
SEED = 0           # integer for repeatable results, None for random
SAVE_PLOT = None   # e.g. "levels.png" to save Figure 1

GRAPH_SIZE = 15    # NSuS: samples in each hill valley graph
MAX_BRANCHES = 4   # NSuS: maximum number of branches (1 = plain SuS)
SCALE = 1.0        # NSuS: proposal std of the Markov chains (None = adaptive)
```

2.2 The performance function

```
def performance_function(x):
    return np.sum(x**2, axis=0)
```

This is the *ring* problem,

$$g(\mathbf{x}) = x_1^2 + x_2^2, \qquad F = \{\mathbf{x} : \|\mathbf{x}\| \geq \sqrt{Y_F}\}, \tag{2}$$

a failure domain with no single most probable point. Since $\|\mathbf{x}\|^2 \sim \chi^2_d$, the exact answer is

$$p_F = P(\chi^2_d \geq Y_F) \overset{d=2}{=} e^{-Y_F/2}, \qquad Y_F = 20 : \; p_F = e^{-10} \approx 4.54 \times 10^{-5}, \tag{3}$$

which the run block computes as `chi2.sf(YF, d)`.

Remark 2.1 (Two input shapes). The function is called with a (d, N) array by subset simulation and the plots, and with a single point by Kinnear's code. Summing over `axis=0` serves both. Kinnear's code passes the point as a Python *tuple* (so that it can be a dictionary key, Section 4.1); the run block therefore hands NSuS the wrapper

`lambda x: performance_function(np.array(x))`.

3 Subset simulation

3.1 Principle

Choose thresholds $Y_1 < Y_2 < \cdots < Y_L < Y_F$ and write $F_k = \{g(\mathbf{x}) > Y_k\}$, so that $F_1 \supset F_2 \supset \cdots \supset F_L \supset F$. Then

$$p_F = P(F_1) \prod_{k=2}^{L} P(F_k \mid F_{k-1}) \; P(F \mid F_L). \tag{4}$$

The thresholds are chosen adaptively so that every factor except the last equals p (here 0.1), which can be estimated accurately from a moderate number of samples.

3.2 Algorithm

Let $n_c = np$ be the number of Markov chains and $n_s = (1-p)/p$ the number of new states per chain, so that $n = n_c(n_s + 1)$; with $n = 3000$, $p = 0.1$ this is $3000 = 300 \times 10$.

Step 1. *Level 0.* Draw $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)} \sim N(\mathbf{0}, \mathbf{I}_d)$, evaluate $y^{(i)} = g(\mathbf{x}^{(i)})$ and count $n_F = \#\{i : y^{(i)} > Y_F\}$.

Step 2. *Stopping rule.* If $n_F/n \geq p$, stop.

Step 3. *Threshold and seeds.* Sort the level so that $y_{(1)} \geq \cdots \geq y_{(n)}$ and set

$$Y_L = \tfrac{1}{2}\big(y_{(n_c)} + y_{(n_c+1)}\big). \tag{5}$$

The n_c samples above Y_L are the *seeds*; they are distributed as $\varphi_d(\cdot \mid F_L)$.

Step 4. *Modified Metropolis.* From each seed $\mathbf{z}^{(0)}$ run n_s steps. Given $\mathbf{z}^{(m)}$, for every coordinate j propose $\xi_j = z_j^{(m)} + \varepsilon_j$, $\varepsilon_j \sim N(0,1)$, and set

$$q_j = \begin{cases} \xi_j & \text{with probability } \min\big(1, \varphi(\xi_j)/\varphi(z_j^{(m)})\big), \\ z_j^{(m)} & \text{otherwise,} \end{cases} \tag{6}$$

$$\mathbf{z}^{(m+1)} = \begin{cases} \mathbf{q} & \text{if } g(\mathbf{q}) > Y_L, \\ \mathbf{z}^{(m)} & \text{otherwise.} \end{cases} \tag{7}$$

The coordinate-wise acceptance preserves φ_d; the second test keeps the chain in F_L.

Step 5. The $n_c(n_s+1) = n$ states form the next level; return to Step 2.

3.3 Estimate and cost

After L conditional levels,

$$\hat{p}_F^{\mathrm{SS}} = p^L \frac{n_F}{n}, \qquad N = n + n(1-p)L. \tag{8}$$

The function returns a dictionary with keys `pF_SS`, `N`, `L`, `Y`, `nF`, `x` and `y`.

Remark 3.1. (i) Only one path is followed, so a failure region whose samples are not among the best $100p\,\%$ at some level is lost. (ii) If all chains are trapped on a local maximum of g below Y_F, the thresholds stop rising and the loop in Step 2 does not terminate; Section 12 gives a guard.

4 The building blocks of NSuS

Kinnear's implementation is a set of small classes. Each is listed below with the file of the `nsus/` package it comes from.

4.1 Cached performance function (`performance_function.py`)

```
def __call__(self, x):
    tuple_x = tuple(x)
    try:
        result = self.cache[tuple_x]
    except KeyError:
        self.cache[tuple_x] = result = self.non_cache_performance_function(tuple_x)
    return result
```

Every evaluation is stored in the dictionary $\mathcal{H}$ (`cache`), the *search history*. The cost reported for NSuS is $N = |\mathcal{H}|$ (`eval_count`), the number of *distinct* points evaluated; repeated states of a Markov chain are free.

4.2 Samples and indicators (`level_create.py`, `indicator.py`)

A `Sample` is a pair $(\mathbf{x}, g(\mathbf{x}))$.

Definition 4.1 (Indicator)**.** An indicator with threshold b and niche function $\mathbb{1}_A$ is

$$I(\mathbf{x}) = \mathbb{1}_A(\mathbf{x})\,\mathbb{1}\{g(\mathbf{x}) \geq b\}. \tag{9}$$

Its method `child_indicator`$(b', \mathbb{1}_{A'})$ returns the indicator $\mathbb{1}_A(\mathbf{x})\mathbb{1}_{A'}(\mathbf{x})\mathbb{1}\{g(\mathbf{x}) \geq b'\}$, and `new_threshold`$(b')$ returns $\mathbb{1}_A(\mathbf{x})\mathbb{1}\{g(\mathbf{x}) \geq b'\}$.

Hence the region of a level at depth m on a branch with niches $A_1, \ldots, A_m$ is

$$R_m = \Big(\bigcap_{k=1}^{m} A_k \Big) \cap \{g \geq b_m\}, \tag{10}$$

so niches nest: a branch never leaves the niches of its ancestors.

4.3 Markov chain (`markov_chain.py`)

`ModifiedMetropolis.update` implements Step 4 of Section 3 in logarithmic form,

$$\text{accept } \xi_j \iff \log \varphi(\xi_j) - \log \varphi(z_j) \geq \log u_j, \quad u_j \sim U(0,1), \tag{11}$$

with proposal standard deviation σ_j supplied by a *scaler*, and with the region test $I(\mathbf{q}) = 1$ in place of $g(\mathbf{q}) > Y_L$. The `ConstantScaler` uses $\sigma_j =$ `SCALE`; the `AdaptiveScaler` (`SCALE = None`) uses the sample standard deviation of the seeds in coordinate j.

4.4 Buds (`bud_create.py`)

Definition 4.2 (Bud)**.** A *bud* is a level that has been specified but not yet sampled: a set of seeds, an indicator I, a scale σ and a parent level. It is *used* once it has been grown into a level.

Given a level L with $|L|$ samples and a niche A, `BudCreator.create` forms

$$\text{candidates } C = \{\mathbf{x} \in L : \not\Vdash_A(\mathbf{x}) = 1\}, \text{ sorted by } g, \tag{12}$$

$$\text{seeds } S = \text{the best } \min\left(\lfloor p|L| \rfloor, |C|\right) \text{ elements of } C, \tag{13}$$

$$\text{threshold } b' = \min_{\mathbf{x} \in S} g(\mathbf{x}). \tag{14}$$

Every level receives a *trivial bud* with $A = \mathbb{R}^d$ (the SuS continuation, used by the stopping rules); a level that is split receives one bud per niche, stored in `_buds`.

4.5 Levels (`level.py`)

A `Level` is a node of an `anytree` tree. Its `name` is the number of levels in the tree at the moment it is created, so names are $1, 2, 3, \ldots$ in creation order. Further members are listed in Table 2.

Member	Meaning
`chain_list`, `sample_list`, `level_size`	samples by chain, flattened, and their number $\lvert L \rvert$
`indicator`	the region R the level was sampled from
`buds`	`_buds` if the level was split, otherwise the trivial bud
`branch`	the path from the root to the level
`stop_info`	which stopping rules fired
`non_used_buds`	all unused buds in the whole tree
`non_used_non_stop_buds`	those whose parent level has not stopped (the options)
`extra_info`	graph, communities and classifier saved by the partitioner

Table 2: Members of `Level`.

4.6 Allocation, choice and stopping

Allocator (`allocate.py`). `SubsetAllocator` uses chains of length $1/p$. `BranchingAllocator`, used by NSuS, divides the budget $\lfloor n(1-p) \rfloor$ among the B buds open at the time:

$$\ell = \left\lfloor \frac{\lfloor n(1-p) \rfloor}{B\,|S|} \right\rfloor + 1, \qquad |L_{\text{new}}| = |S|\,\ell. \tag{15}$$

Chooser (`choose.py`). `DepthChooser` grows an open bud whose parent level has minimal depth. The tree is therefore grown one depth at a time.

Stopping (`stop.py`). With b' the threshold of the trivial bud and b that of the level,

$$\text{Failure: } b' \geq Y_F, \qquad \text{ThresholdConverge: } b' \leq b. \tag{16}$$

The first means at least $\lfloor p|L| \rfloor$ samples already fail; the second means the branch has stopped making progress. A stopped level is the end of its branch.

5 The main loop

```python
def run(self):
    while not self.is_stop:
        if self.is_initial:
            self.initial_level = self.level_creator.create_initial()
            level = self.initial_level
        else:
            bud = self.chooser.choose(self.options)
            level = self.level_creator.create(bud)
        self.bud_creator.create_trivial(level)
        self.stopper.stop(level)
        if not level.is_stop:
            partition_indicators = self.partitioner.partition(level)
            if len(partition_indicators) > 1:
                self.bud_creator.create_inidicator_buds(level, partition_indicators)
    self.print_terminate()
```

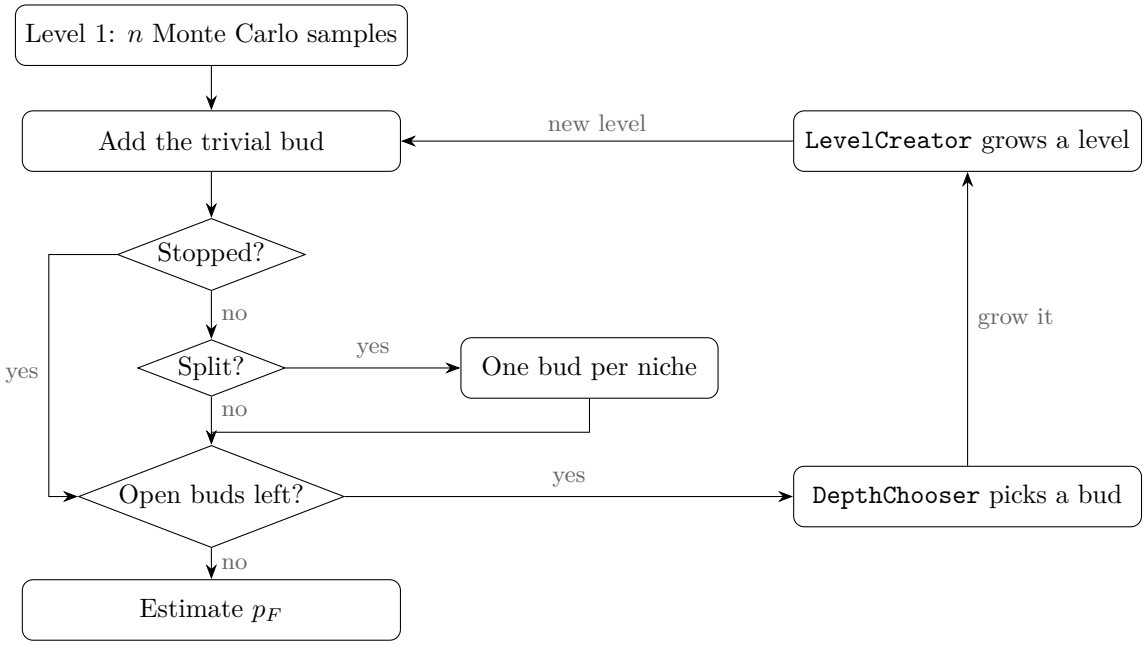


Figure 1: One pass of `NSuS.run` per new level.

Remark 5.1 (Level sizes). By (15), levels are smaller when more buds are open. In the ring run ($n = 3000$): levels 2 and 3 have $|S| = 300$, $B = 3$, $\ell = 4$, size 1200; level 4 has $B = 4$, $\ell = 3$, size 900; all later levels have $|S| \in \{120, 90\}$, $B = 4$, $\ell \in \{6, 8\}$, size 720.

6 The niching step

`HillValleyGraphPartitioner.partition(level)` returns a list of niche functions; a list of length one means *do not split*. It proceeds in five stages.

Stage 0: admissibility. With B the number of open buds in the tree,

$$B \geq \texttt{MAX_BRANCHES} \implies \text{no split}, \qquad k_{\max} = \texttt{MAX_BRANCHES} - B + 1. \tag{17}$$

Stage 1: graph samples. The chains of the level are grouped by their seed; within and across groups the order is randomised, and points are taken one per group in rotation until $m = \lfloor \texttt{GRAPH_SIZE}/B \rfloor$ graph samples $\mathbf{x}_1, \ldots, \mathbf{x}_m$ are chosen.

Stage 2: hill valley graph.

Definition 6.1 (Hill valley test with history)**.** For a pair (i, j) let $Q_{ij} = [\min(\mathbf{x}_i, \mathbf{x}_j), \max(\mathbf{x}_i, \mathbf{x}_j)]$ be the axis-aligned box they span, and $\mathcal{H}_{ij} = \mathcal{H} \cap Q_{ij}$ the stored points inside it, excluding the graph samples. If $\mathcal{H}_{ij} = \emptyset$, g is evaluated once at $(\mathbf{x}_i + \mathbf{x}_j)/2$ and the point is added to $\mathcal{H}$. Then

$$(i, j) \in E \iff g(\mathbf{h}) \geq \min\left(g(\mathbf{x}_i), g(\mathbf{x}_j)\right) \quad \forall\, \mathbf{h} \in \mathcal{H}_{ij}. \tag{18}$$

An edge means no valley was found between the two points: they appear to lie on the same hill of g. The midpoint evaluations are the only additional cost of NSuS (`partitioner.eval_count`).

Stage 3: communities. Asynchronous label propagation is run 100 times on $G = (V, E)$. If more than 50 of the runs return a single community, the level is not split. Otherwise the single-community results are discarded and each remaining partition is reduced to at most $k_{\max}$ parts by randomly merging pairs (`shrink_partition`).

Stage 4: classifier. A pipeline *standardise* $\to$ *PCA* (both fitted on all samples of the level) $\to$ *linear SVC* (ℓ_1 penalty, $C = 1$) is fitted to each candidate partition, labelling graph sample $\mathbf{x}_i$ by its community. The partition with the highest balanced accuracy is kept. The decision boundaries are hyperplanes.

Stage 5: niche functions. For community label k the niche function is

$$\mathbb{1}_{A_k}(\mathbf{x}) = \mathbb{1}\{\text{classifier}(\mathbf{x}) = k\}, \tag{19}$$

and `create_inidicator_buds` creates one bud per niche as in Section 4.

7 Estimation

7.1 The estimator

For a level L and indicator I let $\hat{P}(I, L) = |L|^{-1} \sum_{\mathbf{x} \in L} I(\mathbf{x})$ (`probability_of_indicator`). For a branch $L_0 \to L_1 \to \cdots \to L_m$ with level indicators $I_1, \ldots, I_m$,

$$\hat{p}_{\text{branch}} = \prod_{k=1}^{m} \hat{P}(I_k, L_{k-1}) \cdot \hat{P}(I_m^{Y_F}, L_m), \tag{20}$$

where $I_m^{Y_F}$ is I_m with its threshold replaced by Y_F (`new_threshold`). Without splitting, each factor is approximately p and (20) reduces to $p^L n_F / n$. After a split, each child's factor counts only its own niche, so the factors of sibling branches sum to approximately the factor of an unsplit step. The NSuS estimate is the sum over the leaves of the (trimmed) tree,

$$\hat{p}_F^{\text{NSuS}} = \sum_{\ell \in \text{leaves}} \hat{p}_\ell \qquad (\texttt{exceedance_probability}). \tag{21}$$

For the ring run of Section 9,

$$\hat{p}_F^{\text{NSuS}} = 9.76 \times 10^{-6} + 1.19 \times 10^{-5} + 2.69 \times 10^{-6} + 0 = 2.43 \times 10^{-5}, \tag{22}$$

the zero being the branch stopped by ThresholdConverge.

7.2 Coefficient of variation

With $P_k = \hat{P}(I_k, L_{k-1})$,

$$\delta_1 = \sqrt{\frac{1 - P_1}{n P_1}}, \qquad \delta_k = \sqrt{\frac{1 - P_k}{|L_{k-1}| P_k}\,(1 + \gamma_k)}, \qquad \delta_{\text{branch}} = \Big(\sum_k \delta_k^2 \Big)^{1/2}, \tag{23}$$

where γ_k is the correlation factor of the Markov chains [1], estimated from the auto-correlation of the indicator sequence along each chain (`level_gamma`). The branches are combined as

$$\delta_{p_F} = \Big(\sum_{i,j} w_{ij}\, \delta_{a(i,j)}^2 \Big)^{1/2}, \qquad w_{ij} = \frac{\hat{p}_i \hat{p}_j}{\sum_{i,j} \hat{p}_i \hat{p}_j}, \tag{24}$$

with $a(i,j)$ the last common ancestor of leaves i and j (`cov`). The reliability index is $\beta = -\Phi^{-1}(p_F)$.

7.3 The two assembled algorithms

Component	`SubsetSimulation`	`HillValleyGraphNSuS`
Stopping	Failure, ThresholdConverge	same
Chooser	`DepthChooser`	same
Markov chain	`ModifiedMetropolis`	same
Scaler	constant, or adaptive if `scale=None`	same
Partitioner	`NoPartitioner` (never splits)	hill valley graph, 100 iterations
Classifier	none	ℓ_1 linear SVC, $C = 1$
Allocator	`SubsetAllocator`	`BranchingAllocator`

Table 3: Kinnear's two assemblies of the `NSuS` framework.

Both share a single random generator `np.random.default_rng(seed)`, so a fixed `SEED` reproduces a run exactly.

8 Figures

For $d = 2$ the run block produces seven figures. Those below were produced by the program with the parameters of Section 2; for Figures 5–6 the axis range was widened to $[-4.5, 4.5]^2$ and four niche colours were enabled (Section 10).

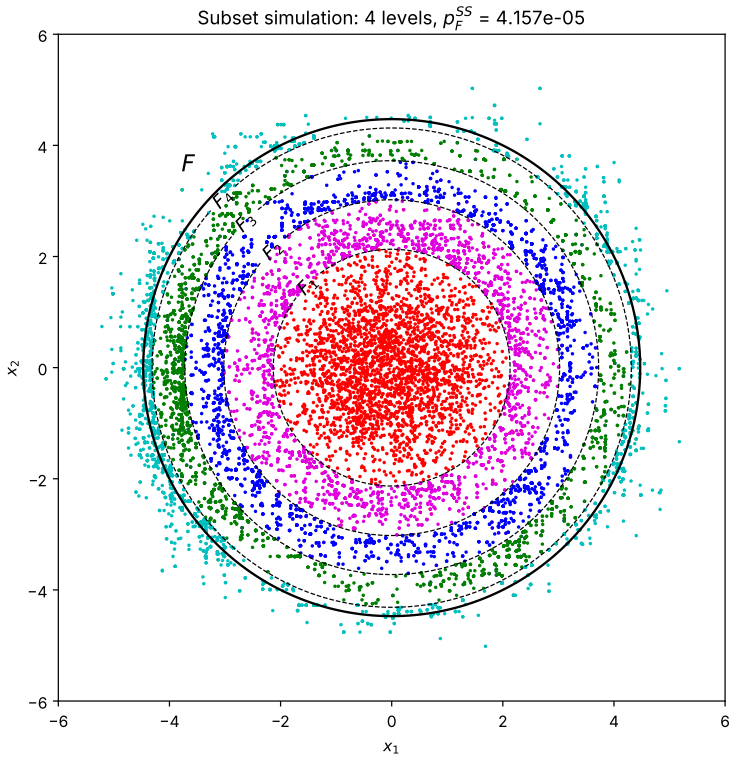


Figure 2: Subset simulation (`plot_levels_2d`). Samples of each level; dashed curves are the intermediate thresholds $\{g = Y_L\}$, labelled F_L; the solid curve is the failure boundary $\{g = Y_F\}$.

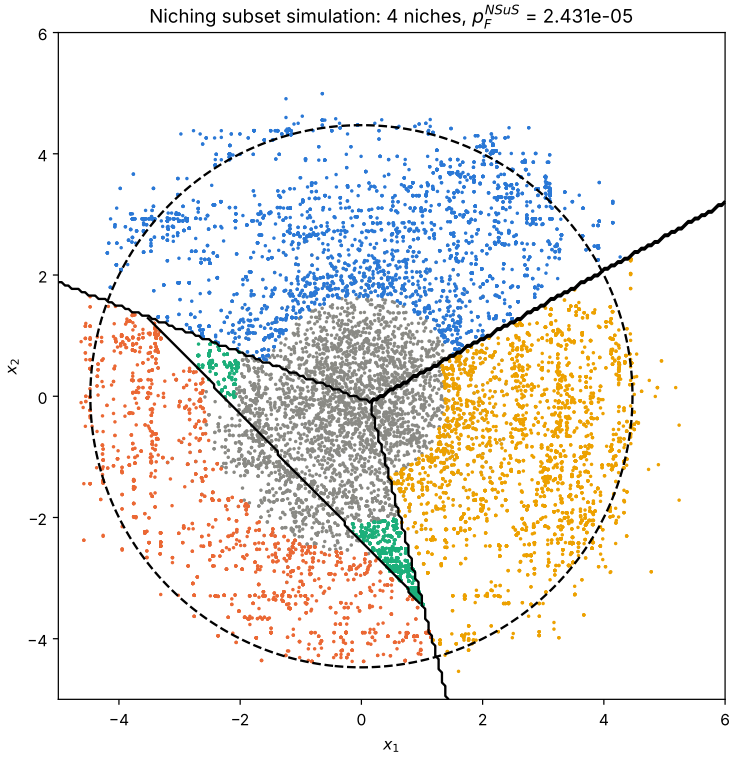


Figure 3: NSuS samples by niche (`plot_nsus_levels_2d`). Grey: levels before the first split; colours: branches. Solid lines: classifier boundaries of each split, drawn only inside the region of the level that was split. Dashed: the failure boundary.

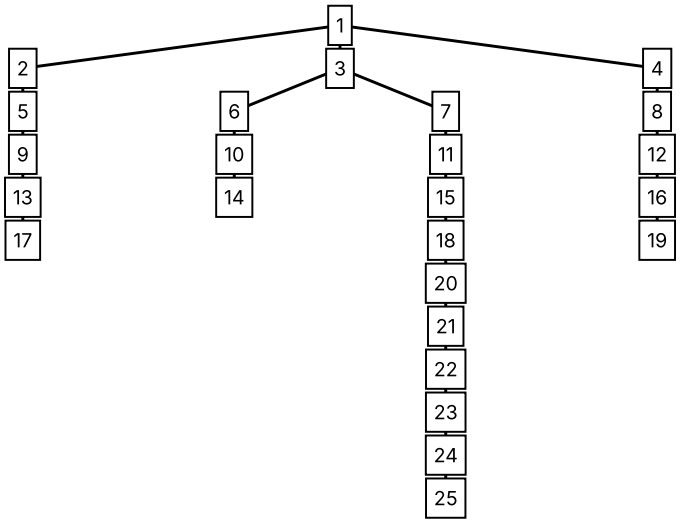


Figure 4: The tree of levels (`plot_branch_tree`). Each box is a level, labelled by its `name`; lines join a level to the levels grown from it. Level 1 splits into 2, 3 and 4, and level 3 into 6 and 7. The labels coincide with those printed by Kinnear's `render_tree`.

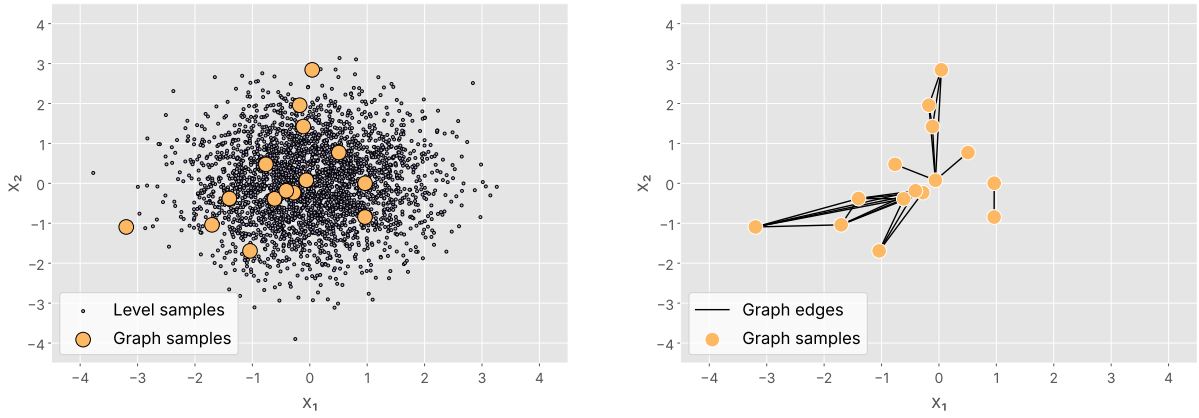


Figure 5: Left: all samples of level 1 and the graph samples chosen in Stage 1. Right: the hill valley graph of Stage 2 (Kinnear's `partitioner.py`).

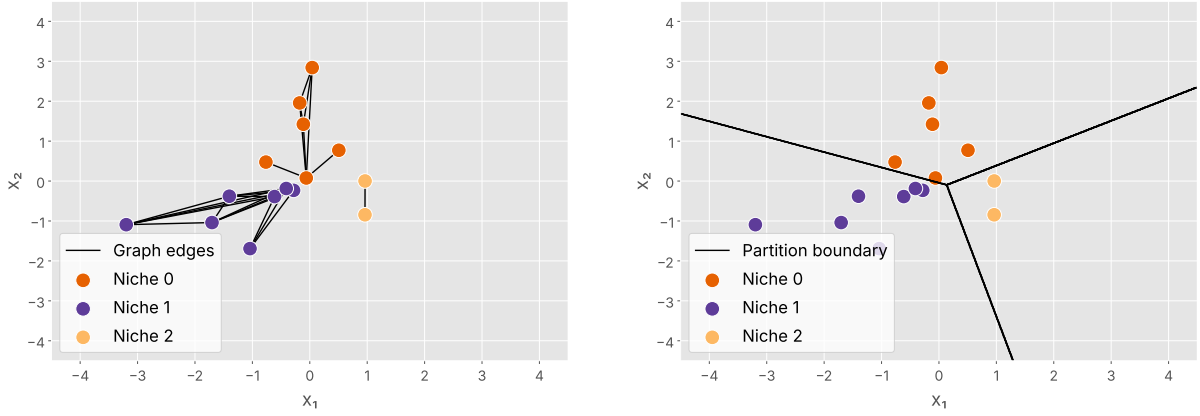


Figure 6: Left: communities found in Stage 3. Right: the partition defined by the classifier of Stage 4.

9 The run block and its output

The run block (i) computes the reference value (3); (ii) runs `subset_simulation`; (iii) builds and runs `HillValleyGraphNSuS` with the parameters of Section 2; (iv) prints $\hat{p}_F$, the number of levels and branches and the cost; (v) draws the figures. For the parameters above the output is

```
SUBSET SIMULATION
Conditional levels L : 4
Total samples N      : 13800
pF (subset sim)      : 4.1567e-05

Level 1 partitioned into 3.
Level 3 partitioned into 2.
Failure stopper has stopped level 14.
Failure stopper has stopped level 17.
Failure stopper has stopped level 19.
Threshold Converge stopper has stopped level 25.
No more options, algorithm terminated.

NICHING SUBSET SIMULATION
Levels / branches    : 25 / 4
Total samples N      : 11644  (0 for hill valley graphs)
pF (NSuS)            : 2.4313e-05

pF (true value)      : 4.5400e-05
```

The costs are counted differently: subset simulation counts $n(1-p)$ new samples per level, NSuS counts distinct evaluations. No midpoint evaluations were needed because, with $n = 3000$, every box Q_{ij} already contained stored points. The ending of each branch can be printed with

```
for leaf in nss.leaves:
    ended = [k for k, v in leaf.stop_info.items() if v]
    print(f"branch ending at level {leaf.name}: {ended}, "
          f"pF contribution = {estimate_leaf_probability(YF, leaf):.3e}")
```

10 Changing the problem and the parameters

10.1 Parameters

Parameter	Value	Effect of increasing it
d	2	higher dimension; figures only for $d = 2$
Y_F	20	rarer event, more levels, smaller p_F
n	3000	lower variance, higher cost; np and $1/p$ must be integers
p	0.1	fewer, larger steps
`SEED`	0	any integer reproduces a run; `None` gives a new run
`GRAPH_SIZE`	15	more reliable niche detection, slightly more cost
`MAX_BRANCHES`	4	more regions followed at once; 1 gives subset simulation
`SCALE`	1.0	larger proposal steps, lower acceptance; `None` is adaptive

Table 4: Parameters and their effect.

10.2 Performance functions

Any g may be used provided it accepts a (d, N) array and a single point, and failure is $g \geq Y_F$; write it with `x[0]`, `x[1]` and NumPy functions. With each new g change Y_F and the reference

value `pF`.

Piecewise linear [2], $Y_F = 0$, reference $p_F = 3.18 \times 10^{-5}$:

$$g(\mathbf{x}) = -\min\big(g_1(x_1), g_2(x_2)\big), \tag{25}$$

$$g_1(x_1) = \begin{cases} 4 - x_1 & x_1 > 3.5, \\ 0.85 - 0.1x_1 & \text{otherwise}, \end{cases} \qquad g_2(x_2) = \begin{cases} 0.5 - 0.1x_2 & x_2 > 2, \\ 2.3 - x_2 & \text{otherwise}. \end{cases} \tag{26}$$

```
def performance_function(x):
    g1 = np.where(x[0] > 3.5, 4 - x[0], 0.85 - 0.1 * x[0])
    g2 = np.where(x[1] > 2, 0.5 - 0.1 * x[1], 2.3 - x[1])
    return -np.minimum(g1, g2)
```

Mixture of four bumps [2], $Y_F = 0$, reference $p_F = 2.19 \times 10^{-4}$:

$$g(\mathbf{x}) = \frac{1}{2\pi} \sum_{i=1}^{4} w_i \exp\Big(-\tfrac{1}{2}\|\mathbf{x} - \boldsymbol{\mu}_i\|^2 \Big) - 0.04, \tag{27}$$

with $\boldsymbol{\mu} = (3,3), (2,-2), (-2,2), (-2,-2)$ and $w = (0.4, 0.2, 0.2, 0.2)$.

```
def performance_function(x):
    x = np.asarray(x, dtype=float)
    means = [(3, 3), (2, -2), (-2, 2), (-2, -2)]
    weights = [0.4, 0.2, 0.2, 0.2]
    total = 0
    for w, (m1, m2) in zip(weights, means):
        total = total + w * np.exp(-0.5 * ((x[0] - m1)**2 + (x[1] - m2)**2))
    return total / (2 * np.pi) - 0.04
```

Functions with exact answers.

$$\begin{aligned} &g = x_1 + x_2, && Y_F = 5, && p_F = \Phi(-5/\sqrt{2}), \\ &g = |x_1|, && Y_F = 3.5, && p_F = 2\Phi(-3.5), \\ &g = \max(|x_1|, |x_2|), && Y_F = 3.5, && p_F = 1 - \big(1 - 2\Phi(-3.5)\big)^2. \end{aligned} \tag{28}$$

10.3 Figures for a new problem

Kinnear's `partitioner_figures` uses the fixed range $[-2, 3.2] \times [-4, 4]$ unless one is passed, e.g.

```
partitioner_figures(nss, split[0].name, x_range=(-4.5, 4.5), y_range=(-4.5, 4.5))
```

To colour up to four niches, replace the `zip` line in both places by

```
for part_set,c,label in zip(sample_partition,[colors[0],colors[3],colors[1],colors
    [2]],['Niche 0','Niche 1','Niche 2','Niche 3']):
```

11 Numerical experiments

11.1 Ring, $Y_F = 20$, $p_F = 4.54 \times 10^{-5}$

Change from Section 2	$\hat{p}_F^{\mathrm{NSuS}}$	N	Levels	Branch endings
none	2.43×10^{-5}	11644	25	F F X F
`MAX_BRANCHES = 1`	3.78×10^{-5}	12452	5	F
`MAX_BRANCHES = 2`	5.05×10^{-5}	11770	9	F F
`GRAPH_SIZE = 30`	3.04×10^{-5}	11425	26	X F F F
`SCALE = None`	4.02×10^{-5}	11946	19	F F F F
$n = 1000$	2.77×10^{-5}	3823	20	F F F F

Table 5: Single runs, `SEED = 0`. F: branch reached failure; X: stopped by ThresholdConverge.

Y_F	exact p_F	$\hat{p}_F^{\mathrm{SS}}$	$\hat{p}_F^{\mathrm{NSuS}}$
20	4.54×10^{-5}	4.16×10^{-5}	2.43×10^{-5}
25	3.73×10^{-6}	2.80×10^{-6}	1.14×10^{-6}
30	3.06×10^{-7}	1.82×10^{-7}	8.50×10^{-8}

20 seeds, $n = 1000$	mean $\hat{p}_F$	CoV	mean N
SuS	4.99×10^{-5}	0.78	4465
NSuS, 1 branch	4.39×10^{-5}	0.51	4118
NSuS, 4 branches	4.75×10^{-5}	0.74	3779

Table 6: Left: rarer events, `SEED = 0`, $n = 3000$. Right: repeated runs.

The ring is a single connected failure region. NSuS nevertheless splits it into angular sectors, each sector receives a smaller share of the budget, and one sector frequently stalls. Over repeated runs all three estimators are close to the exact value with comparable scatter: niching offers no advantage here.

11.2 Piecewise linear, $Y_F = 0$, $p_F = 3.18 \times 10^{-5}$

Change from Kinnear's example	$\hat{p}_F^{\mathrm{NSuS}}$	N (hill valley)	Levels / branches
none	1.83×10^{-5}	2421 (6)	10 / 2
`MAX_BRANCHES = 1`	3.18×10^{-7}	2881 (0)	7 / 1
`GRAPH_SIZE = 6`	5.72×10^{-5}	2567 (2)	11 / 2
`GRAPH_SIZE = 30`	5.94×10^{-5}	2440 (38)	10 / 2
`SCALE = None`	4.48×10^{-5}	2990 (5)	12 / 2
`MAX_BRANCHES = 4`	3.97×10^{-5}	2174 (10)	19 / 4

Table 7: Kinnear's example: $n = 500$, `SEED = 2`, `GRAPH_SIZE = 14`, `MAX_BRANCHES = 2`.

Without splitting the run follows only the more accessible failure region and misses the region $\{x_1 \geq 4, x_2 \leq 2\}$, underestimating p_F by two orders of magnitude. With two branches both regions are followed. Over 100 runs Kinnear reports that subset simulation misses this region in 65% of runs and NSuS in 2% [2]. Niching is beneficial when the failure domain consists of genuinely separate regions.

12 Troubleshooting

Symptom	Remedy
`TypeError` in `ModifiedMetropolis.__init__`	a class was pasted inside another; each class must appear once
`NameError` for a function	save the file; keep every `def` above the run block
`NameError: plt`	keep `import matplotlib.pyplot as plt` at the top
tuple errors inside g	pass `lambda x: performance_function(np.array(x))`
`ValueError: n*p and 1/p must be integers`	use n a multiple of 10 for $p = 0.1$
program never finishes	guard subset simulation: `while nF[L] / n < p and L < 30:`
`RecursionError`	`sys.setrecursionlimit(10000)`
points outside Figures 5–6	pass `x_range`, `y_range`
`KeyError` in `partitioner_figures`	name a level that was split
$\hat{p}_F = 0$	no branch reached Y_F: increase n or `MAX_BRANCHES`, or change `SEED`

References

[1] S.-K. Au and J. L. Beck, Estimation of small failure probabilities in high dimensions by subset simulation, *Probabilistic Engineering Mechanics* 16(4):263–277, 2001.

[2] H. Kinnear, *Niching subset simulation*, with companion code at `https://github.com/HughKinnear/nsus_paper`.